

AI Fitness Memory Agent System Architecture

How CockroachDB, AWS, and a model-agnostic LangGraph agent interact to make persistent, evidence-grade memory the product rather than a feature.

The architectural thesis. Health memory is mostly **typed quantitative events**, and the questions people actually ask are **computations**: *“show my protein during June”* is a `SUM(payload->>'protein_g') ... GROUP BY week`, not a similarity search. But *“what did I say about my knee?”* genuinely is one. A system that answers both, over the same rows, with no window in which its computed and remembered views of the user disagree, needs **SQL aggregation and vector search in one transactionally consistent store**. That requirement selected CockroachDB, and it is why swapping the database out would not be a port — it would be a redesign. Every claim the agent makes resolves back to the memory rows that produced it, and the UI shows the user the executed queries. AWS runs the rest: Bedrock produces the embeddings that fill the vector column, and ECS Express Mode, ECR, Secrets Manager, IAM, and CloudWatch carry the application from a push on `main` to a live HTTPS URL.

SUBMISSION

CockroachDB × AWS
Agentic Memory Hackathon

COCKROACHDB TOOLS

Distributed Vector Indexing
(runtime)
ccloud CLI (cluster
provisioning)

AWS SERVICES

Bedrock · ECS Express Mode
(Fargate) · ECR
Secrets Manager ·
CloudWatch · IAM · ALB

STACK

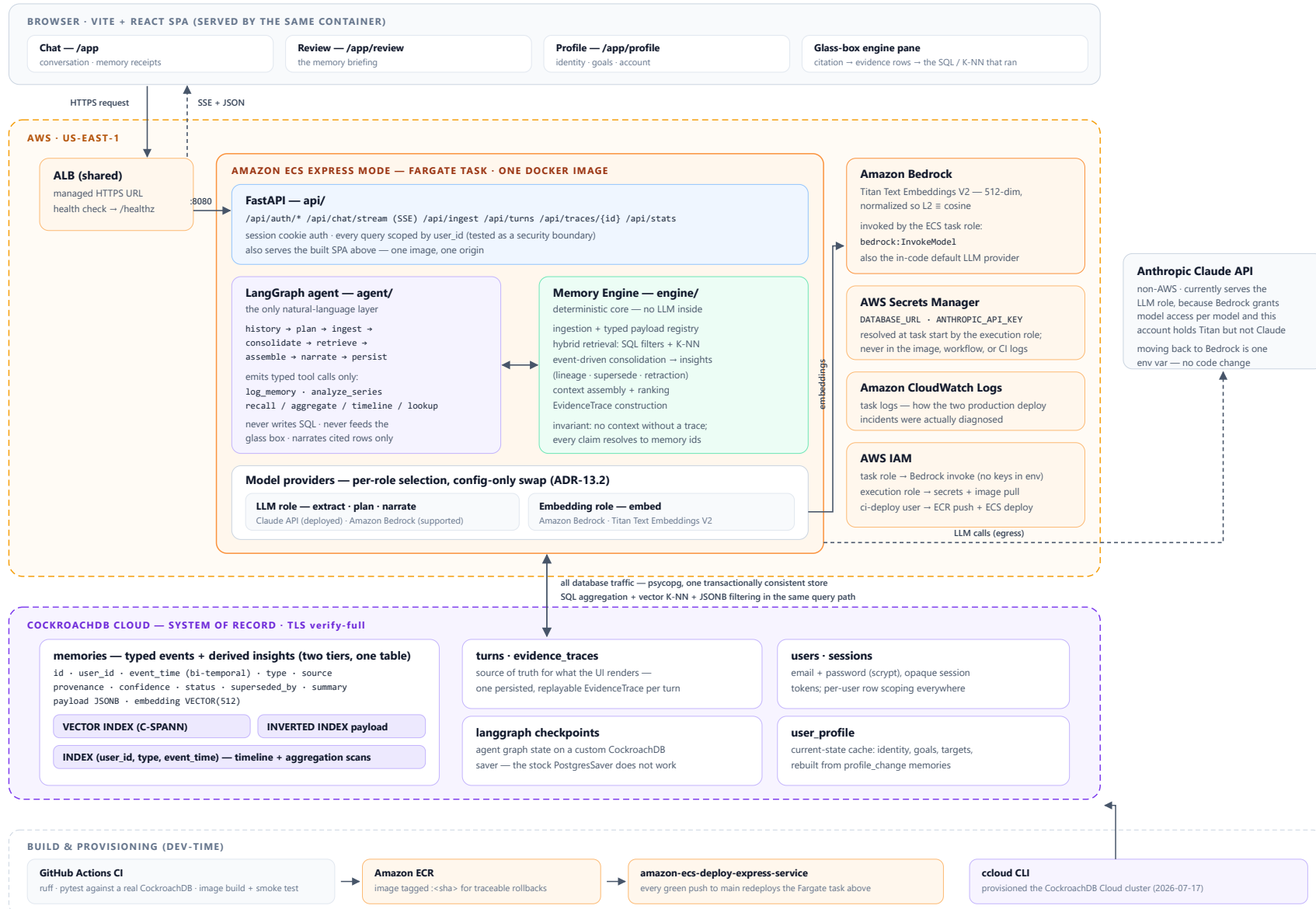
FastAPI · LangGraph ·
psycopy 3
React 19 + Vite · one Docker
image

CONTENTS

1 Overview · 2 System map ·
3 Turn lifecycle
4 CockroachDB · 5 AWS · 6
Memory model

AI Fitness Memory Agent — System Architecture

Lifelong memory as the product: typed JSONB + VECTOR(512) in CockroachDB Cloud, a deterministic Memory Engine, and a model-agnostic LangGraph agent running on AWS.



03 Anatomy of a turn

One question, end to end — where each component and service enters



- 1 Request arrives**
The SPA calls POST /api/chat/stream. The shared **ALB** terminates HTTPS and forwards to the Fargate task on port 8080. FastAPI resolves the session cookie to a user_id; from here every query in the turn is scoped to it.
- 2 history — short-term memory**
The recent messages of this thread are read from the turns table. They reach **plan** and **narrate** only. History is never evidence: never citable, never inside an EvidenceTrace, and never an ingestion source.
- 3 plan — the only free-form step**
The LLM sees the question, the short-term history, and the tool vocabulary, and returns **typed tool calls**. It never writes SQL. Calls are validated into typed specs before any statement is composed; an unknown tool or unparseable slot fails here, not in the database.
- 4 ingest — the write path**
log_memory is a **zero-argument signal**: it says whether this turn is loggable, never what to log. The engine extracts typed events from the bytes of the current request only — at most once per turn — validates each against the payload registry, and embeds them.
- 5 embed — Amazon Bedrock**
Titan Text Embeddings V2 returns a 512-dim vector per memory, normalized so that L2 distance is equivalent to cosine. Bedrock is invoked with the ECS **task role** — no keys in the environment. If embedding fails, rows are stored with NULL embeddings and stay eligible for backfill rather than being given fabricated vectors.

- 6 consolidate — derived insights**
A write touching a series triggers detection on that series only, synchronously, under a time budget. Confirmed level shifts and intervention outcomes are written back as **insight** memories carrying **lineage** to the events that produced them, and are retracted or superseded when the evidence beneath them changes.
- 7 retrieve — hybrid, deterministic**
Each validated spec becomes one parameterised statement: SQL aggregation, timeline scans, and JSONB lookups run against the composite and inverted indexes; semantic recall runs K-NN against the vector column. Same store, same transaction boundary, same instant.
- 8 assemble — context + trace together**
Ranked context is built **and** the EvidenceTrace is constructed in the same pass: the executed SQL with its bound parameters, row counts, evidence snapshots, insight references, ranking, and the exact set of memory IDs the narrator is permitted to cite. The invariant is structural — **no context without a trace**.
- 9 narrate — constrained language**
The LLM writes prose over retrieved rows and cites them by id. It cannot reach the database, and it cannot feed the glass box: the trace was already built from what actually executed.
- 10 persist — replayable truth**
The turn and its trace are written to turns and evidence_traces; graph state goes to the checkpoint tables. Reopening the conversation weeks later re-renders the same citations from stored rows, not from a re-run of the model.

What the user sees while this happens. Each stage is streamed over SSE as it actually occurs — extracting → analyzing → retrieving → assembling context → generating. The engine pane is a live view of the real graph, not a progress animation. If the stream cannot be established, the client falls back to the plain endpoint automatically.

One table, two memory tiers

Episodic events and derived insights share the `memories` table. Structure lives in a typed JSONB payload rather than rigid nutrition columns, so a new nutrient tomorrow is a new key, not a migration. Time is bi-temporal: `event_time` (when it happened, possibly estimated) is separate from `created_at` (when we learned it), which is what lets the system answer questions about the past honestly.

```
-- engine/schema.sql
CREATE TABLE memories (
  id          UUID PRIMARY KEY,
  user_id     UUID NOT NULL,
  event_time  TIMESTAMPTZ, tz TEXT,
  type        TEXT, source TEXT,
  provenance  TEXT, confidence FLOAT,
  status      TEXT, superseded_by UUID,
  summary     TEXT,
  payload     JSONB NOT NULL,
  embedding   VECTOR(512),
  created_at  TIMESTAMPTZ,
  VECTOR INDEX memories_embedding_idx (embedding),
  INVERTED INDEX memories_payload_idx (payload),
  INDEX memories_user_type_time_idx
    (user_id, type, event_time)
);
```

- meal
- workout
- sleep
- body_scan
- weight
- blood_report
- supplement
- note
- profile_change
- insight

Three indexes, three questions

INDEX	SERVES
VECTOR INDEX (C-SPANN)	Distributed Vector Indexing. Semantic recall — “what did I say about my knee?” — as K-NN over normalized 512-dim vectors.
INVERTED INDEX (payload)	Containment and key filtering inside JSONB, which is what makes the migration-free payload design usable rather than merely flexible.
INDEX (user_id, type, event_time)	Timeline reconstruction and every aggregation — the computational questions that no vector store can answer.

```
-- semantic recall (engine/retrieval.py)
SELECT id, type, event_time, confidence,
       provenance, summary,
       embedding <-> %(qvec)s::VECTOR(512) AS distance
FROM memories
WHERE user_id = %(user_id)s
      AND status = 'active'
      AND embedding IS NOT NULL
ORDER BY embedding <-> %(qvec)s::VECTOR(512)
LIMIT %(top_k)s
```

Statements are always parameterised — builder families never interpolate values — which is what makes the executed SQL safe to display verbatim in the glass box.

The rest of the cluster

TABLE	ROLE
turns	Conversation history; the short-term memory source and the UI's record of what was said.
evidence_traces	One persisted, replayable trace per turn — the glass box reads stored rows, never a re-run.
users · sessions	Email + password (scrypt), opaque session tokens, per-user row scoping tested as a security boundary.
user_profile	Current-state cache — identity, goals, targets — rebuilt from <code>profile_change</code> memories.
langgraph checkpoints	Agent graph state on a custom CockroachDB saver; only small <code>serde-safe</code> channels are ever checkpointed.

ccloud CLI

The CockroachDB Cloud cluster was provisioned with the `ccloud` CLI. The application connects with `sslmode=verify-full` against a CA baked into the image, and CI runs the full test suite against a real CockroachDB on every push, so the schema and both canaries can never silently rot.

Measured, not assumed. The vector index is real, created, and asserted by a permanent CI canary — including that the plan uses it. At the product's current query shape, the *filtered* per-user K-NN falls back to a scan, because this CockroachDB version abandons the vector index under residual filters. The filters are correctness boundaries (user scoping, superseded-row exclusion), so they stay; the index becomes load-bearing exactly when per-user row counts outgrow a scan — the lifelong-memory case the product is designed for.

Amazon Bedrock — the intelligence layer

Titan Text Embeddings V2 produces the 512-dim normalized vector for every memory written. Normalization is an invariant, not a detail: C-SPANN is Euclidean-only, so unit vectors are what make L2 distance equivalent to cosine similarity. Bedrock is therefore the direct input to the CockroachDB vector column — remove it and semantic recall has nothing to search.

Bedrock is also the **in-code default LLM provider** for extraction, planning, and narration. The deployed LLM role currently runs on the Claude API because Bedrock grants model access per model and this account holds Titan but not Claude inference; the two roles were made independently selectable for exactly this reason. Moving the LLM role back to Bedrock is one environment variable and no code change — the acceptance test for the model-independence contract.

Amazon ECS Express Mode — the runtime

One Docker image containing FastAPI and the built React SPA runs as a Fargate task behind a shared ALB with a managed HTTPS URL, health-checked at `/healthz` on port 8080. One image and one origin means no CORS surface, no second deployment target, and no drift between the API a judge tests and the UI they see.

AWS IAM — three identities, deliberately separated

Task role	<code>bedrock:InvokeModel</code> — assumed by the running container, so no credentials exist in the environment.
------------------	--

Execution role	Pulls the image and resolves secrets at task start.
-----------------------	---

CI user	ECR push, ECS Express deploy, and scoped <code>PassRole</code> — nothing else.
----------------	--

AWS Secrets Manager — configuration that never leaks

The database URL and the model API key are ECS secrets entries resolved by the execution role at task start, so their values never appear in the image, the workflow file, the task definition, or CI logs. Everything non-sensitive is a literal in the version-controlled workflow, and a preflight step fails the job if a required variable is unset — a half-configured service never reaches the ALB.

Amazon ECR + delivery

Every push to `main` runs lint and the full test suite against a real CockroachDB, builds the image, and smoke-tests it. Green builds are pushed to ECR tagged by commit SHA — so a rollback is a tag, not an archaeology exercise — and deployed by the official `amazon-ecs-deploy-express-service` action. There is no manual deploy step.

Amazon CloudWatch Logs — the debugging surface

Task logs are how two real production incidents were diagnosed rather than guessed at: a `verify-full` connection failing for want of a root certificate path inside the image, and a stored secret carrying a trailing newline that the HTTP stack rejected as an illegal header — surfacing as a generic “Connection error” indistinguishable from a network outage without the full exception chain.

Deliberately absent. No S3 — an uploaded meal photo is decoded, validated, EXIF-stripped, sent to the vision model, and discarded; nothing is persisted, so there is no image store to secure. **No Lambda, no queue, no scheduler** — consolidation runs synchronously inside the ingestion request under a time budget. Both are stated architectural decisions: no infrastructure for completeness.

Two memories, structurally separated

Short-term

The recent messages of the current thread, read from turns. Answers “*what are we talking about right now?*” It reaches **plan** and **narrate** only — never citable, never in a trace, never an ingestion source.

Long-term

Typed rows in *memories*, retrieved deterministically and cited by id. Answers “*what do I know about this user?*” This is the product.

The separation is enforced by construction, not convention: the log signal carries no payload, and ingestion reads the bytes of the current request at most once per turn. The write path has no dedupe, so a fact re-ingested from an earlier turn would become a permanent duplicate that silently inflates every aggregate over its range while the glass box cites it as genuine evidence. A dedicated test suite, whose providers misbehave on purpose, exists to catch any regression here.

The agent's closed vocabulary

The planner may emit these calls and nothing else. Read tools are structurally incapable of writing; write tools never pass through the query builders.

aggregate_memories

recall_memories

get_timeline

lookup_events

count_events

lookup_insights

log_memory

analyze_series

Honesty rules the system enforces

- **No context without a trace.** Assembly builds both or neither — asserted as a property test.
- **Citations must be in the permitted set.** The trace carries the exact citable IDs; a citation outside it is invalid by construction.
- **Estimated values stay marked.** Provenance and confidence travel with every row and are shown, not averaged away.
- **Failure is reported, not hidden.** If semantic recall is unavailable, the turn degrades honestly and says so instead of answering as though it had searched.
- **Superseded rows never resurface.** Corrections supersede rather than overwrite, so history stays auditable.

What the glass box actually shows

Clicking a citation resolves it to the rows that produced it, and “how this was retrieved” shows the executed statements themselves.

TRACE FIELD	CONTENTS
retrieval_steps	The exact SQL with placeholders, its bound parameters, and the number of memory rows that participated.
evidence	Snapshots of the rows retrieved, with type, event time, provenance, and confidence.
insights	References to derived insights, each traceable to its lineage.
timeline	The reconstructed sequence backing temporal claims.
ranking	Why each item made it into context.
citable_ids	Exactly what the narrator was permitted to cite — including aggregate contributors that never appear as snapshots.

Why this is the interesting part. An AI coach grounded in your own data is now table stakes. Resolving a claim back to the specific rows that produced it — and showing the query that found them — is what a deterministic engine over a single consistent store makes possible, and what a model asked to remember on its own cannot offer.